MPI - CheatSheet

Compiled BY : Manthan A. D. [in]

Ch. 1 - INTRO. TO MICROPROCESSORS

1. Essential terminology

Term	Definition
Microprocessor (MPU)	Programmable chip that executes instructions and processes data.
ALU	Arithmetic Logic Unit; performs arithmetic and logical operations.
Register	Small, fast storage location inside the processor.
Control Unit	Directs instruction execution and coordinates operations.
Timing Unit	Generates timing/control signals to synchronize operations.
Instruction	Binary-coded command executed by the processor.
Opcode	Instruction field specifying the operation.
Operand	Data or location on which an instruction operates.
System bus	Signal lines connecting CPU, memory and I/O devices.

2. System buses

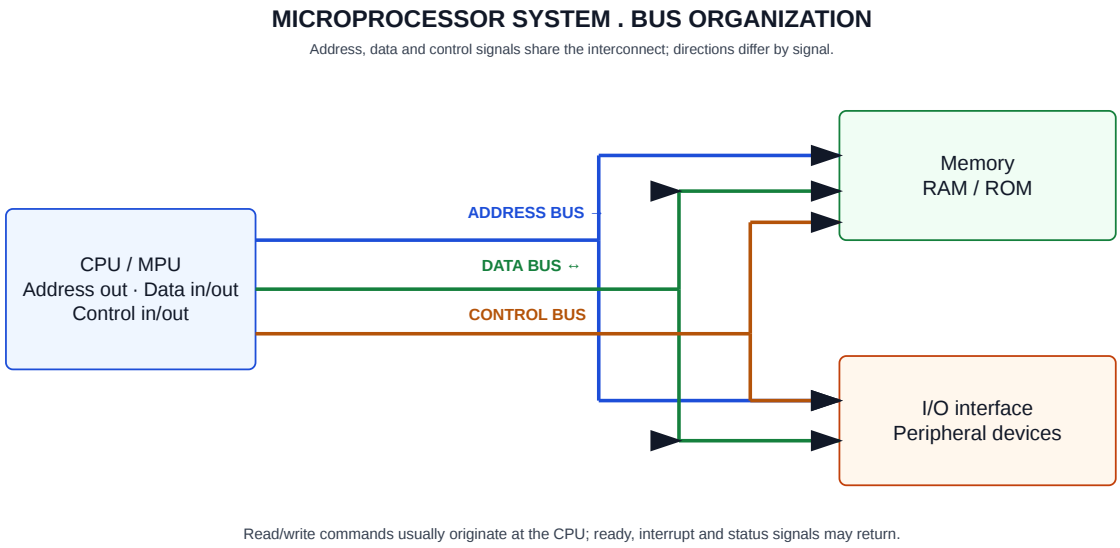
Bus	Purpose	Direction
Address bus	Specifies memory location or I/O port	Generally CPU to device
Data bus	Carries instructions and data	Bidirectional
Control bus	Carries read, write, interrupt and timing signals	Depends on signal

Address capacity: An n-bit address bus can represent 2^n unique addresses.

$$\text{Addressable locations} = 2^n$$

Example: 16 address bits = 65,536 byte locations = 64 KB.

3. Basic microprocessor system



Direction cue: Address normally travels from CPU to device; data is bidirectional; control-signal directions vary.

The CPU fetches instructions/data from memory, processes them, and communicates with external devices through the system bus.

4. Assembly vs machine language

Assembly language	Machine language
Uses mnemonics such as `MOV A,B`	Uses binary instruction codes
Easier for humans to read	Directly executed by the processor
Converted by an assembler	Executed by the processor

- **Assembler:** Converts assembly instructions into machine code.
- **Debugger:** Helps locate and correct program errors.

Ch. 2 - ARCHITECTURE, MEMORY AND I/O

1. Essential terminology

Term	Meaning
Memory	Stores programs and data.
RAM	Random Access Memory; read/write and generally volatile.
ROM	Read Only Memory; non-volatile storage.
I/O	Input/Output; communication with external devices.
Port	Addressable interface used to communicate with an I/O device.
Memory read	Processor reads data from a memory location.
Memory write	Processor writes data to a memory location.

2. Read and write operations

- **Read:** Processor places an address on the address bus, activates read control, and receives data.
- **Write:** Processor places address and data on their buses and activates write control.

3. M.IMP - Memory-mapped I/O vs I/O-mapped I/O

Feature	Memory-mapped I/O	I/O-mapped / Peripheral-mapped I/O
Address space	Shares memory address space	Separate I/O address space
8085 address size	Uses part of 64 KB memory space	8-bit port address
Address capacity	Depends on memory map	Up to 256 port addresses
Instructions	Normal memory instructions	`IN` and `OUT`
Control signals	`MEMR`, `MEMW`	`IOR`, `IOW`

Feature	Memory-mapped I/O	I/O-mapped / Peripheral-mapped I/O
Advantage	Uses normal memory operations	Preserves memory address space

Exam focus: Explain address-space distinction, instructions, control signals and at least three differences.

4. I/O device examples

- **Input:** Keyboard, switches, sensors.
- **Output:** Display, LEDs, printer.
- **Bidirectional:** Devices that both send and receive data.

Ch. 3 - 8085 ARCHITECTURE, PINS, BUSES AND TIMING

1. Essential terminology

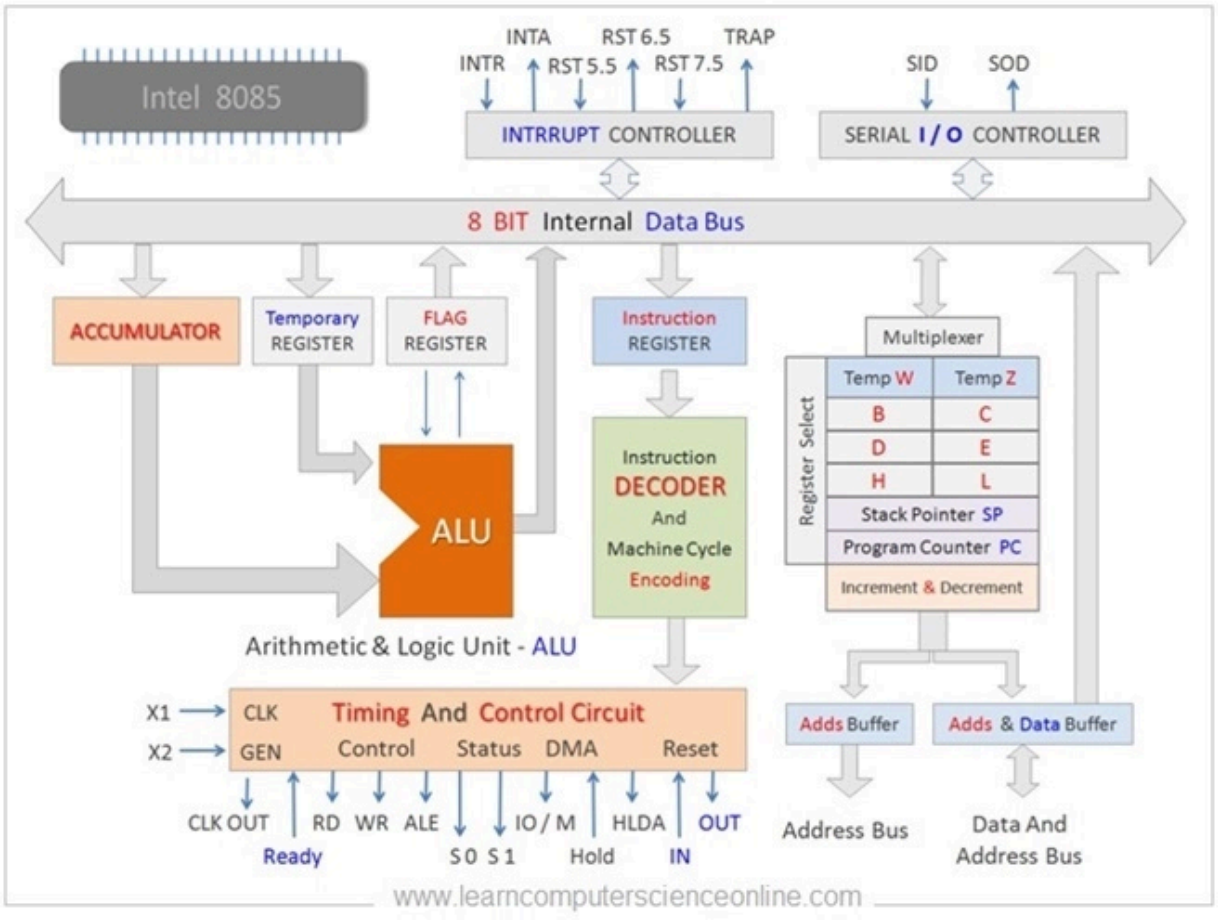
Term	Definition
8085	8-bit microprocessor.
Accumulator (A)	8-bit register used in many ALU operations.
Flag register	Stores status information about results.
Program Counter (PC)	16-bit register holding the address of the next instruction to fetch.
Stack Pointer (SP)	16-bit register pointing to the top of the stack.
Instruction Register (IR)	Temporarily holds the fetched instruction for decoding.
Multiplexed bus	Same physical lines carry different information at different times.
ALE	Address Latch Enable; indicates low-order address on AD0-AD7.
Machine cycle	Basic bus-operation cycle, such as opcode fetch or memory read/write.

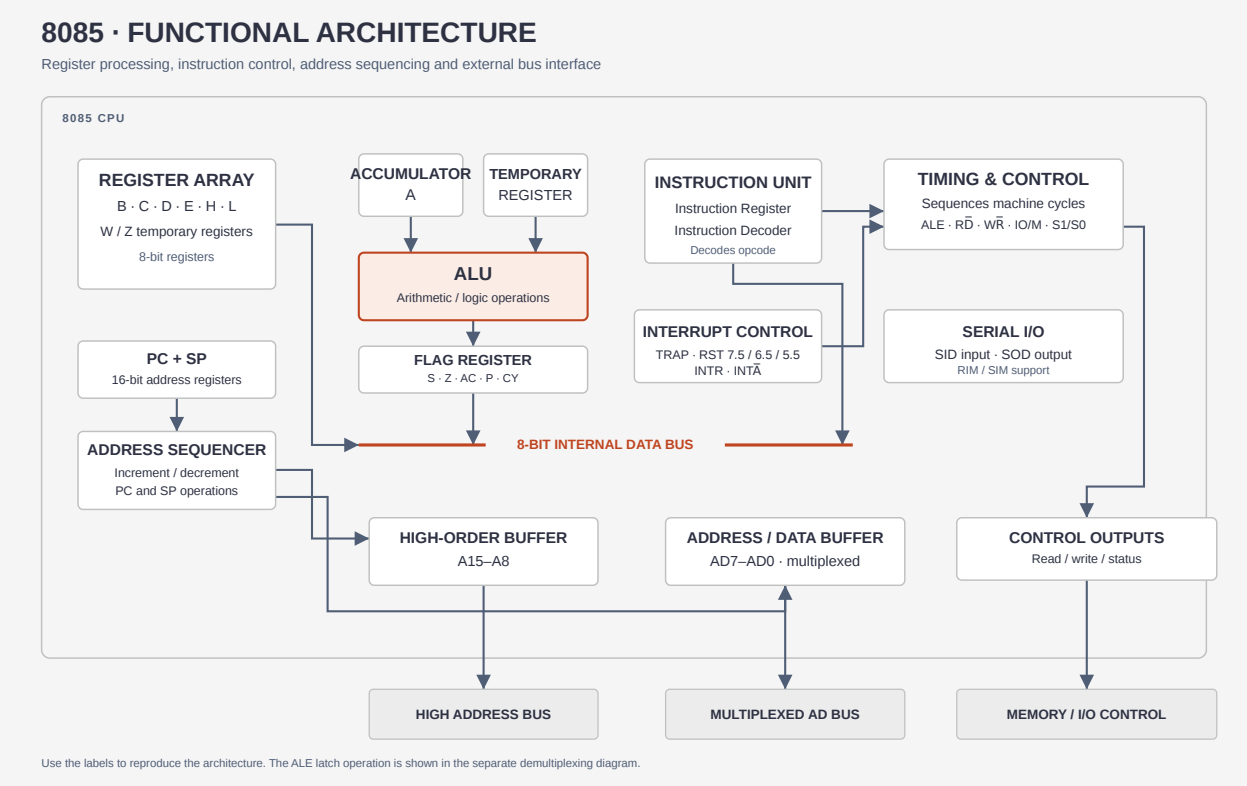
Term	Definition
T-state	One processor clock period.

2. M.IMP - 8085 specifications

- 8-bit processor and ALU.
- 16-bit address bus; 8-bit data bus.
- 64 KB addressable memory space.
- 40-pin IC.
- Six main general-purpose registers: B, C, D, E, H, L.
- Five main condition flags: S, Z, AC, P, CY.
- Hardware interrupts: TRAP, RST 7.5, RST 6.5, RST 5.5 and INTR.

3. M.IMP - 8085 internal architecture


Intel 8085
INTA
INTR
RST 5.5
RST 6.5
RST 7.5
TRAP
INTRRUPT CONTROLLER
SID
SOD
SERIAL I / O CONTROLLER
8 BIT Internal Data Bus
ACCUMULATOR
Temporary REGISTER
FLAG REGISTER
Instruction REGISTER
ALU
Arithmetic & Logic Unit - ALU
Instruction DECODER And Machine Cycle Encoding
Multiplexer
Register Select
Temp W
Temp Z
B
C
D
E
H
L
Stack Pointer SP
Program Counter PC
Increment & Decrement
X1
X2
CLK GEN
Timing And Control Circuit
Control
Status
DMA
Reset
CLK OUT
Ready
RD
WR
ALE
S0
S1
IO / M
Hold
HLDA
IN
OUT
Adds Buffer
Adds & Data Buffer
Address Bus
Data And Address Bus
www.learncomputerscienceonline.com



Selected 8085 functional architecture

Diagram note: This is a clear functional revision diagram, not a literal gate-level circuit. Practise the labelled structure and bus connections.

Exam redraw checklist: register array B, C, D, E, H, L; W/Z temporary registers; accumulator + ALU + flags; instruction register/decoder; timing/control; PC and SP; increment/decrement address circuitry; separate A15-A8 address buffer and AD7-AD0 address/data buffer; interrupt control; serial I/O. Keep the two external buffers separate.

4. M.IMP - Programming model

Register	Size	Purpose
A	8-bit	Accumulator
B, C, D, E, H, L	8-bit each	General-purpose registers
BC, DE, HL	16-bit pairs	16-bit data or addresses
PC	16-bit	Next-instruction address
SP	16-bit	Stack pointer

Register	Size	Purpose
IR	8-bit internal register	Holds current instruction for decoding
Flags	5 main flags	Record result conditions

Important: `M` is not a physical register. In `MOV A,M`, it means the memory location addressed by the HL pair.

5. M.IMP - Flag register

Flag	Bit	Set condition
S - Sign	D7	Result's most significant bit is 1
Z - Zero	D6	Result is zero
AC - Auxiliary Carry	D4	Carry from bit 3 to bit 4
P - Parity	D2	Result contains an even number of 1-bits
CY - Carry	D0	Carry out of most significant bit; in subtraction, indicates borrow

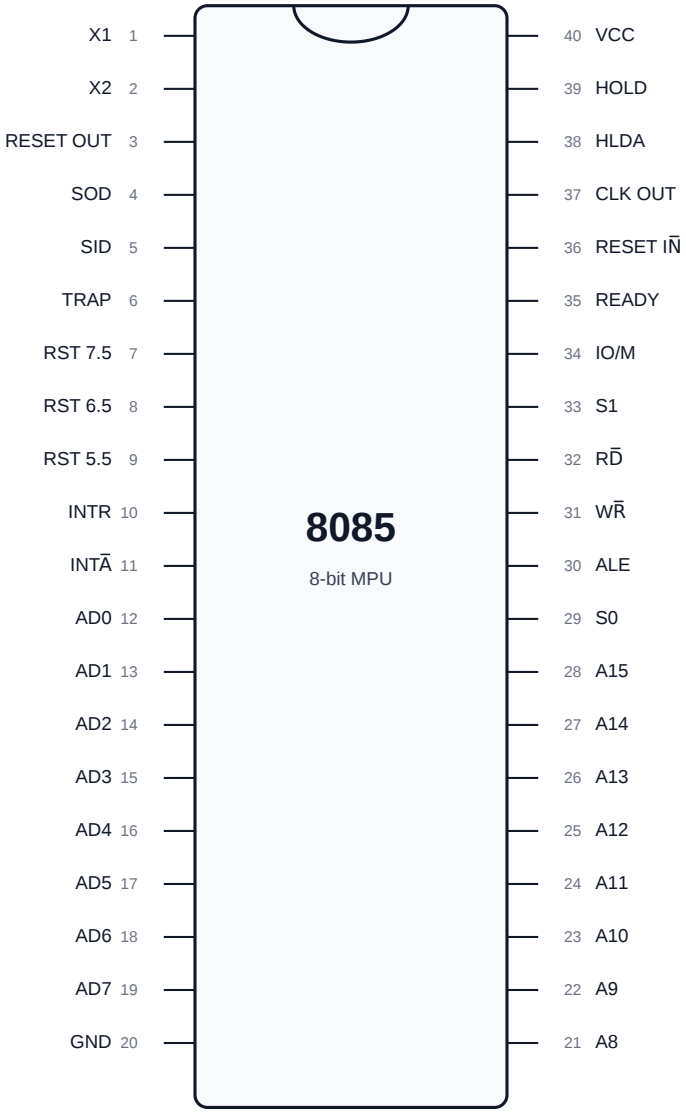
Conventional flag-register layout:

```
D7  D6  D5  D4  D3  D2  D1  D0
 S   Z   0  AC   0   P   1  CY
```

6. M.IMP - 8085 pin diagram

8085 MICROPROCESSOR . 40-PIN DIP PINOUT

Top view. Pin 1 starts at upper left; numbering continues down left, then up right.



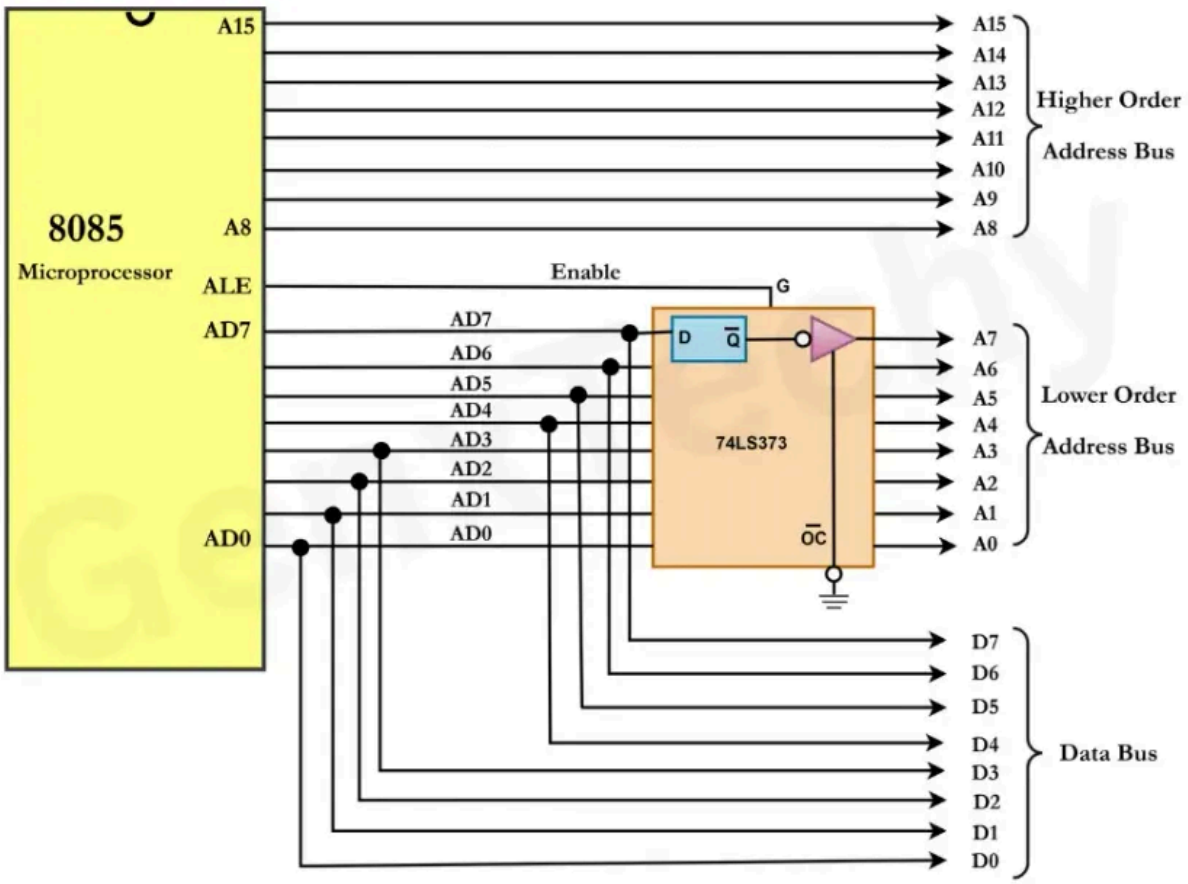
Overbar indicates active-low. Check exact line notation against your course reference.

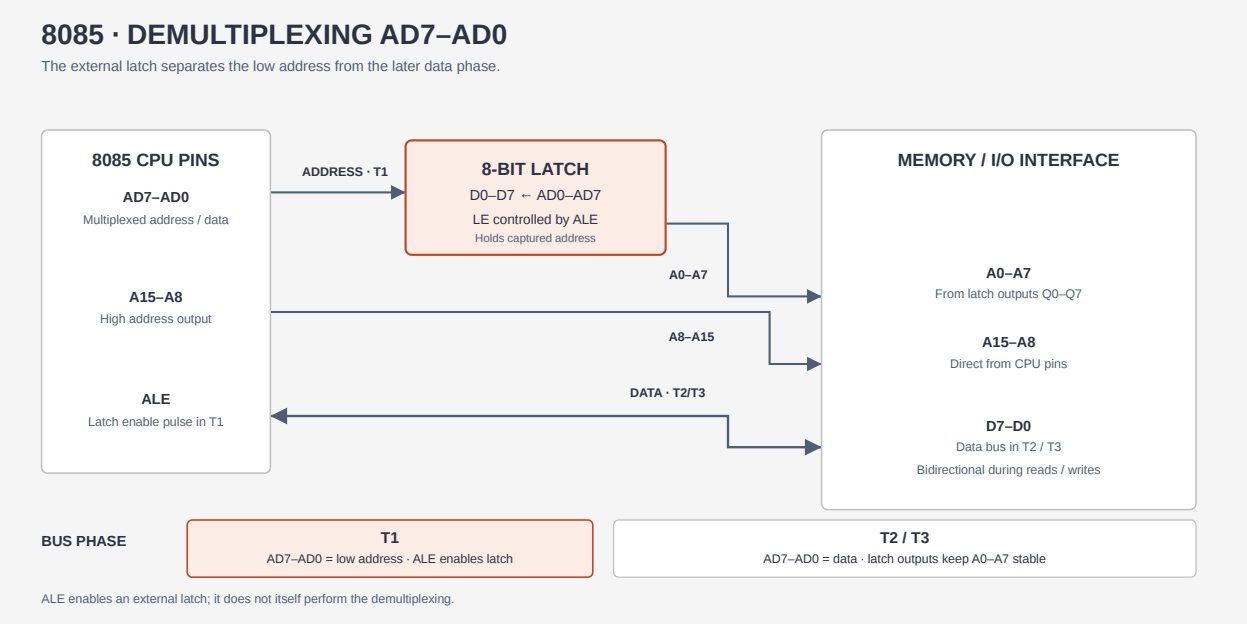
`*` means active-low.

Pin/signal	Function
AD0-AD7	Multiplexed low-order address/data bus
A8-A15	High-order address bus
ALE	Enables latch to capture low-order address

Pin/signal	Function
RD*	Read control signal
WR*	Write control signal
IO/M	0 = memory operation; 1 = I/O operation
READY	Allows processor to wait for slower device
HOLD / HLDA	Bus request / bus-grant acknowledgement
INTR / INTA*	General maskable interrupt request / acknowledgement
TRAP	Highest-priority, non-maskable hardware interrupt
RST 7.5, 6.5, 5.5	Maskable vectored hardware interrupts
SID / SOD	Serial input / serial output data
RESET IN*	Resets processor
RESET OUT	Indicates processor reset condition to external circuitry

7. M.IMP - Demultiplexing AD0-AD7





Selected 8085 AD7–AD0 demultiplexing

1. During T1, AD0-AD7 carry the low-order address.

2. ALE enables the latch to capture that address.
3. The latch keeps A0-A7 stable.
4. AD0-AD7 can then carry data during the rest of the operation.

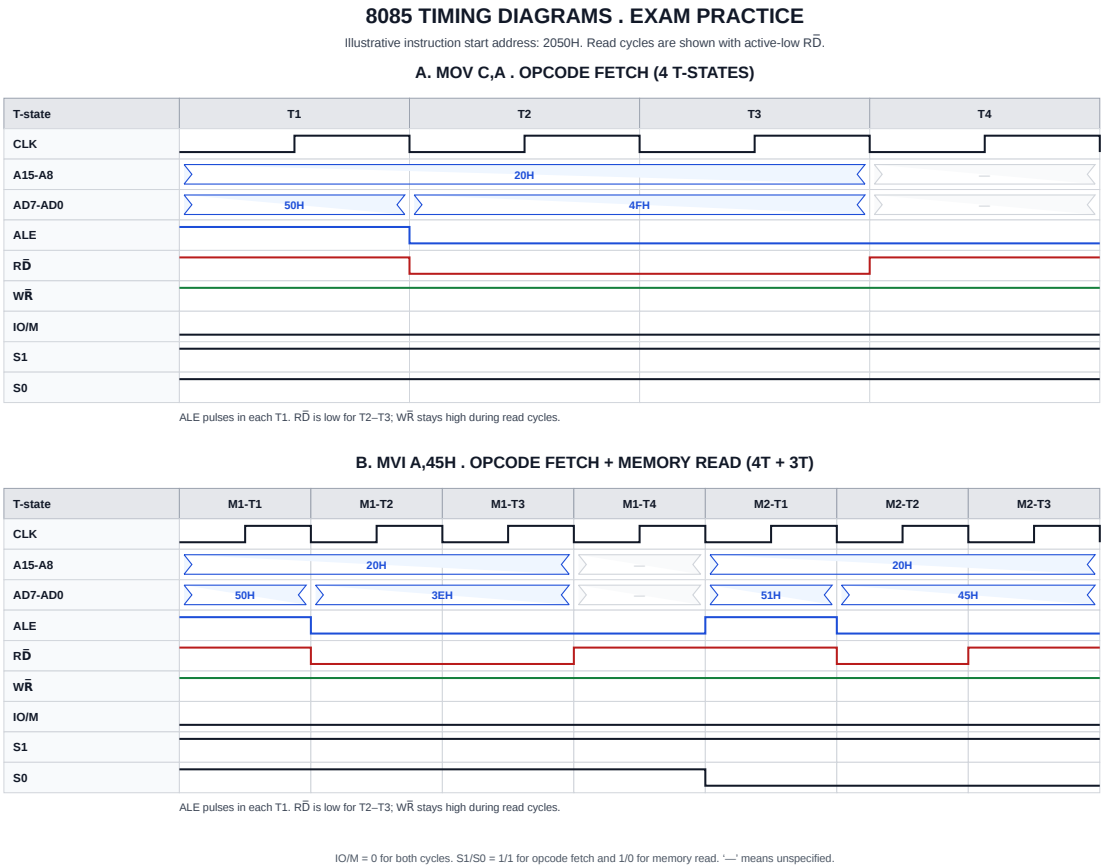
A15-A8 carry the upper address directly.

8. M.IMP - Instruction cycle, machine cycle and T-state

Term	Meaning
T-state	One clock period
Machine cycle	Bus operation, such as opcode fetch or memory read
Instruction cycle	Total time required to fetch and execute an instruction

Instruction	Bytes	Machine cycles	T-states
`MOV C,A`	1	1	4
`MVI A,45H`	2	2	7
`MOV M,D`	1	2	7
`ADD M`	1	2	7

`MOV M,D` is one byte but needs a memory-write cycle. `ADD M` needs a memory-read cycle. Instruction length and machine-cycle count are different concepts.



How to draw it: T1 carries the address and ALE pulses high. During T2-T3, RD̄ goes low for the read. IO/M = 0 throughout both memory cycles. S1/S0 = 1/1 for opcode fetch and 1/0 for memory read. AD7-AD0 changes from low address to opcode/data; A15-A8 carries the high address through T3.

Reference for the standard timing phases: University of Delhi, Microprocessor 8085 . Timing and Control.

9. M.IMP - Memory interfacing and address decoding

- **Address decoding:** Determines which memory chip should respond to an address.
- **Chip Select (CS):** Enables the required chip.
- **Absolute/full decoding:** Checks all required address bits; avoids unintended address aliases.

- **Partial decoding:** Checks only some address bits; may cause address mirroring/foldback.

4 KB EPROM using 74LS138:

- 4 KB = 4096 = 2^12 locations; the chip uses A0-A11 internally.
- Higher-order address bits identify the intended address range.
- 74LS138 is a 3-to-8 decoder that can generate chip-select signals.
- RAM and EPROM must have non-overlapping assigned address ranges.

Exam drawing checklist: A0-A7 are latched from AD0-AD7 using ALE; A8-A11 connect directly to a 4 KB device's upper address pins; A12-A15 are used in chip-select decoding. Use 74LS138 outputs (active-low) for chip selects, connect read control to $\overline{OE}$, and write control to RAM $\overline{WE}$. Derive the actual address map and enable logic from the addresses given in the question. The exact circuit cannot be fixed without that address range.

10. Control-signal generation

IO/M	RD* active	WR* active	Operation
0	Yes	No	Memory read - MEMR
0	No	Yes	Memory write - MEMW
1	Yes	No	I/O read - IOR
1	No	Yes	I/O write - IOW

Read and write signals are active-low. IO/M distinguishes memory from I/O operations.

Ch. 4 - 8085 ASSEMBLY LANGUAGE AND INSTRUCTION SET

1. Essential terminology

- **ALP:** Assembly Language Program, written using assembly mnemonics.

- **Mnemonic:** Symbolic instruction name, such as `ADD`.
- **Addressing mode:** Method used to identify an operand.
- **Assembler:** Converts assembly language into machine code.
- **Instruction format:** Layout of opcode and operand fields.

2. M.IMP - Addressing modes

Mode	Meaning	Example
Immediate	Data is part of the instruction	`MVI A,32H`
Register	Operand is in a register	`MOV A,B`
Direct	Memory address supplied explicitly	`LDA 2050H`
Register-indirect	Register pair holds memory address	`MOV A,M` • address in HL
Implied	Operand is implied	`CMA`

3. Instruction and data formats

Format	Layout	Example
1-byte	Opcode	`MOV A,B`
2-byte	Opcode + 8-bit data	`MVI A,45H`
3-byte	Opcode + 16-bit address/data	`LXI H,2050H`

For 16-bit addresses and immediate data, the 8085 instruction stream generally stores the low-order byte before the high-order byte.

4. Essential 8085 instruction command table

A. Data transfer

Command	Function	Example
`MOV Rd,Rs`	Copy register to register	`MOV A,B`
`MVI R,data`	Load 8-bit immediate data	`MVI A,25H`
`LXI RP,data16`	Load 16-bit register pair	`LXI H,2050H`
`LDA addr`	Load A from direct memory	`LDA 2050H`

Command	Function	Example
`STA addr`	Store A to direct memory	`STA 2050H`
`LHLD addr`	Load L and H from consecutive memory bytes	`LHLD 2050H`
`SHLD addr`	Store L and H to consecutive memory bytes	`SHLD 2050H`
`LDAX RP`	Load A indirectly through BC or DE	`LDAX B`
`STAX RP`	Store A indirectly through BC or DE	`STAX D`
`XCHG`	Exchange HL and DE	`XCHG`

B. Arithmetic

Command	Function
`ADD R/M`	Add operand to A
`ADC R/M`	Add operand and carry to A
`ADI data`	Add immediate data to A
`SUB R/M`	Subtract operand from A
`SBB R/M`	Subtract operand and borrow from A
`SUI data`	Subtract immediate data from A
`INR R/M`	Increment by 1
`DCR R/M`	Decrement by 1
`INX RP` / `DCX RP`	Increment/decrement register pair
`DAD RP`	Add register pair to HL
`DAA`	Decimal Adjust Accumulator after BCD addition
`CMP R/M` / `CPI data`	Compare with A; A is unchanged

C. Logical and rotate operations

Command	Function
`ANA R/M` / `ANI data`	AND with A
`ORA R/M` / `ORI data`	OR with A
`XRA R/M` / `XRI data`	XOR with A
`CMA`	Complement all bits in A

Command	Function
`CMC`	Complement carry flag
`STC`	Set carry flag
`RLC` / `RRC`	Rotate A left/right with outgoing bit returned to A
`RAL` / `RAR`	Rotate A left/right through carry

RLC/RRC vs RAL/RAR: RLC and RRC rotate the outgoing bit back into A and CY. RAL and RAR rotate through CY, so the previous CY enters A.

D. Branching and control

Command	Function
`JMP addr`	Unconditional jump
`JZ` / `JNZ`	Jump if zero / not zero
`JC` / `JNC`	Jump if carry / no carry
`JP` / `JM`	Jump if positive / minus
`JPE` / `JPO`	Jump if even / odd parity
`CALL addr`	Call subroutine
`RET`	Return from subroutine
`RST n`	Call fixed restart vector
`PCHL`	Load PC from HL
`NOP`	No operation
`HLT`	Halt normal instruction execution

E. Stack, I/O and interrupt control

Command	Function
`PUSH RP`	Save register pair on stack
`POP RP`	Restore register pair from stack
`IN port`	Read input port into A
`OUT port`	Send A to output port
`EI` / `DI`	Enable/disable maskable interrupts

Command	Function
`RIM` / `SIM`	Read interrupt/serial status / set masks and serial output
`SPHL`	Copy HL to SP
`XTHL`	Exchange HL with top two stack bytes

Flag caution: Instructions do not all update flags. `MOV` does not change flags. `INR` / `DCR` update S, Z, AC and P but do not change CY.

Ch. 5 - 8085 PROGRAMMING WITH DECISIONS AND LOOPS

1. Program-writing method

1. Identify where data is stored and where result must go.
2. Load starting address or initial data.
3. Perform the operation using registers and ALU.
4. Use conditional jumps for decisions and loops.
5. Store or output the result.
6. Terminate with `HLT` when appropriate.

2. M.IMP - Find largest of N numbers

Assumption: Count at 2050H, data starts at 2051H, result stored at 2060H. Array has at least one byte.

```
LXI H,2050H
MOV C,M
INX H
MOV A,M
DCR C
JZ DONE
```

```
LOOP: INX H
CMP M
JNC NEXT
MOV A,M

NEXT: DCR C
JNZ LOOP

DONE: STA 2060H
HLT
```

Logic: Keep current maximum in A. `CMP M` compares A with the current memory byte. If A is greater than or equal to it, `JNC` skips the update; otherwise replace A.

Smallest number: After `CMP M`, skip the update when `JC` or `JZ` is true. Update A when current A is greater than the memory byte.

3. M.IMP - Bubble sort N bytes

Assumption: Count at 2050H, array starts at 2051H, N >= 2.

```
LDA 2050H
MOV B,A
DCR B

OUTER: MOV C,B
LXI H,2051H

INNER: MOV A,M
INX H
CMP M
JC NOSWAP
JZ NOSWAP

MOV D,M
DCX H
MOV M,D
```

```
INX H
MOV M,A

NOSWAP: DCR C
JNZ INNER
DCR B
JNZ OUTER
HLT
```

- **Ascending:** Keep `JC NOSWAP` and `JZ NOSWAP`.
- **Descending:** Replace those two instructions with `JNC NOSWAP`.

The inner loop compares adjacent bytes. The outer loop repeats passes to complete the sort.

4. M.IMP - Block transfer

Assumption: Source starts at 2100H, destination at 2200H, count at 2000H.

```
LXI H,2100H
LXI D,2200H
LDA 2000H
MOV C,A
MOV A,C
ORA A
JZ DONE

LOOP: MOV A,M
STAX D
INX H
INX D
DCR C
JNZ LOOP

DONE: HLT
```

HL points to source, DE to destination, and C contains byte count. Each iteration copies one byte and increments both pointers.

5. IMP - Add two numbers and check carry

Assumption: Operands at 2050H and 2051H; output to port 01H. If carry occurs, output 01H instead of sum.

```
LDA 2050H
MOV B,A
LDA 2051H
ADD B
JNC DISPLAY
MVI A,01H

DISPLAY: OUT 01H
HLT
```

6. Other programs to practise

Problem	Main method
Largest / smallest in array	`CMP`, conditional jump, loop counter
Ascending / descending sort	Nested loops and swapping
Copy N bytes	`MOV`, `STAX`, `INX`, `DCR`, `JNZ`
Two's complement	`CMA`, `ADI 01H`
Positive / negative separation	Test sign bit and store accordingly
Sum with carry check	`ADD`, then test CY
Loop/counter program	Counter register and conditional jump

Two's-complement template:

```
LDA 2050H
CMA
ADI 01H
```

```
STA 2051H
HLT
```

Ch. 6 - STACK, SUBROUTINES, DELAY AND CODE CONVERSION

1. Essential terminology

Term	Meaning
Stack	Last-In, First-Out (LIFO) storage area in memory
Stack Pointer (SP)	Points to the top of the stack
Subroutine	Reusable block of code called from another program location
Counter	Register/memory value repeatedly changed to count operations
Time-delay routine	Loop whose execution time produces a delay
BCD	Binary-Coded Decimal; each decimal digit represented by four bits
Code conversion	Conversion between representations such as BCD, binary and ASCII

2. M.IMP - Stack and subroutines

- The 8085 stack grows toward lower memory addresses.
- `PUSH` saves a register pair; `POP` restores it.
- `PUSH PSW` saves A and the flag register; `POP PSW` restores them.
- `CALL` transfers execution to a subroutine and saves the return address.
- `RET` restores that return address and resumes the caller.
- Initialize SP and reserve valid stack memory before stack-dependent code runs.

PUSH / POP	CALL / RET
Save/restore register contents	Transfer control to/from subroutine
Operate on register pair or PSW	Manage subroutine return address

```
CALL DELAY
HLT

DELAY:
; delay-loop instructions
RET
```

3. M.IMP - Delay-loop calculation

$$t_{\text{state}} = 1/f_{\text{clock}}$$

$$T_{\text{delay}} = N_{\text{states}} \times t_{\text{state}}$$

$$T_{\text{delay}} = N_{\text{states}}/f_{\text{clock}}$$

Exam method: Determine execution count of each instruction, calculate total T-states including loop branches and counter operations, then multiply by T-state duration. Do not estimate delay from loop count alone.

4. BCD and conversion essentials

- **Packed BCD:** Two decimal digits occupy one byte, one digit per nibble.
- **Unpacked BCD:** One decimal digit is stored per byte.
- **DAA:** Adjusts accumulator after BCD addition.
- **Two's complement:** Invert all bits and add 1.
- **Hex to ASCII:** Digits 0-9 use ASCII 30H-39H; letters A-F use 41H-46H.

Example: Decimal 45 has packed BCD 45H, while its binary value is 2DH. These are different representations.

5. 16-bit data operations

Instruction	Operation
`LXI RP,data16`	Load register pair with 16-bit data

Instruction	Operation
`INX RP`	Increment register pair
`DCX RP`	Decrement register pair
`DAD RP`	Add register pair to HL
`LHLD addr`	Load 16 bits from consecutive memory locations into HL
`SHLD addr`	Store HL to consecutive memory locations

Remember: `DAD` affects the carry flag; do not assume every flag is updated by every 16-bit operation.

Ch. 7 - INTERFACING, INTERRUPTS, 8259A AND 8255A

1. Essential terminology

Term	Definition
Interfacing	Connecting processor with memory or peripherals compatibly
Interrupt	Event requesting processor attention outside normal program flow
Vectored interrupt	Service-routine address is predetermined
Maskable interrupt	Can be disabled by interrupt mask/control
Non-maskable interrupt	Cannot be disabled by normal interrupt masking
PIC	Programmable Interrupt Controller
PPI	Programmable Peripheral Interface

2. M.IMP - 8085 interrupts

Interrupt	Vector address	Priority	Maskable?	Type
TRAP	0024H	Highest	No	Hardware, vectored

Interrupt	Vector address	Priority	Maskable?	Type
RST 7.5	003CH	2	Yes	Hardware, vectored
RST 6.5	0034H	3	Yes	Hardware, vectored
RST 5.5	002CH	4	Yes	Hardware, vectored
INTR	Not fixed	Lowest	Yes	Hardware, non-vectored

Priority: TRAP > RST 7.5 > RST 6.5 > RST 5.5 > INTR.

Software interrupts: RST 0-RST 7 have restart vectors at 0000H, 0008H, 0010H, 0018H, 0020H, 0028H, 0030H and 0038H.

3. RIM and SIM

RIM - Read Interrupt Mask: Reads serial input and interrupt status, including pending interrupts, interrupt-enable status and mask bits.

RIM bit	Meaning
D7	SID
D6, D5, D4	Pending RST 7.5, 6.5, 5.5
D3	Interrupt-enable status
D2, D1, D0	Mask status for RST 7.5, 6.5, 5.5

SIM - Set Interrupt Mask: Controls interrupt masks and serial output.

SIM bit	Meaning
D7	Serial output data (SOD)
D6	Serial Data Enable (SDE)
D5	Don't care
D4	Reset RST 7.5 pending latch
D3	Mask Set Enable (MSE)
D2, D1, D0	Mask values for RST 7.5, 6.5, 5.5

With MSE = 1, mask bits are applied. A mask bit of 1 masks the corresponding interrupt.

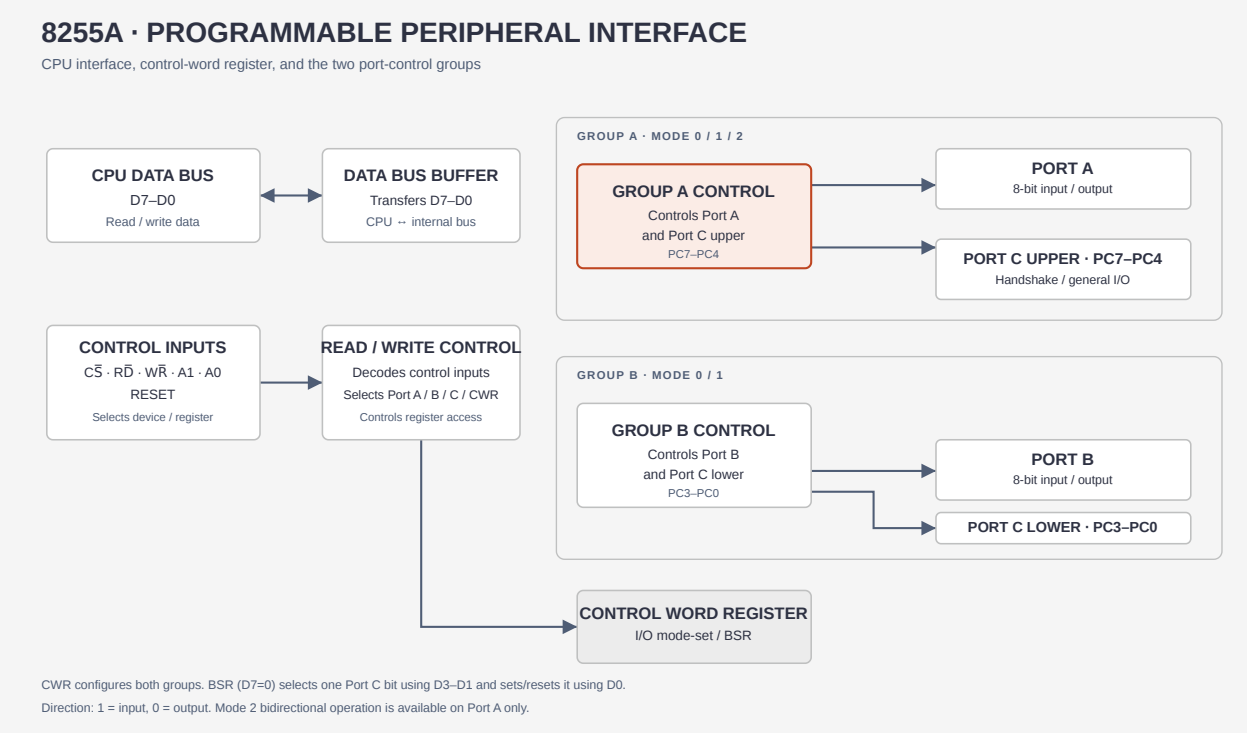
4. M.IMP - 8255A Programmable Peripheral Interface

8255A provides three 8-bit ports:

- **Port A:** 8-bit port.
- **Port B:** 8-bit port.
- **Port C:** 8-bit port, also usable as two 4-bit groups.

Port C upper and lower halves can provide control/handshake signals.

Mode	Name	Purpose
Mode 0	Simple I/O	Basic input/output without handshake
Mode 1	Strobed/handshaked I/O	Uses handshake signals
Mode 2	Bidirectional bus	Bidirectional, handshaked operation on Port A
BSR	Bit Set/Reset	Individually sets/resets a Port C bit



Selected 8255A block diagram

I/O mode-set control word

Bit	Function
D7	1 = I/O mode-set format
D6-D5	Group A mode selection
D4	Port A direction
D3	Port C upper direction
D2	Group B mode selection
D1	Port B direction
D0	Port C lower direction

Direction bits: 1 = input, 0 = output. For mode-set control word, D7 = 1.

BSR mode: D7 = 0. D3-D1 select a Port C bit; D0 selects set (1) or reset (0). BSR words modify the selected Port C bit.

5. M.IMP - 8259A Programmable Interrupt Controller

Purpose: Manages interrupt requests, prioritizes them, and signals the processor.

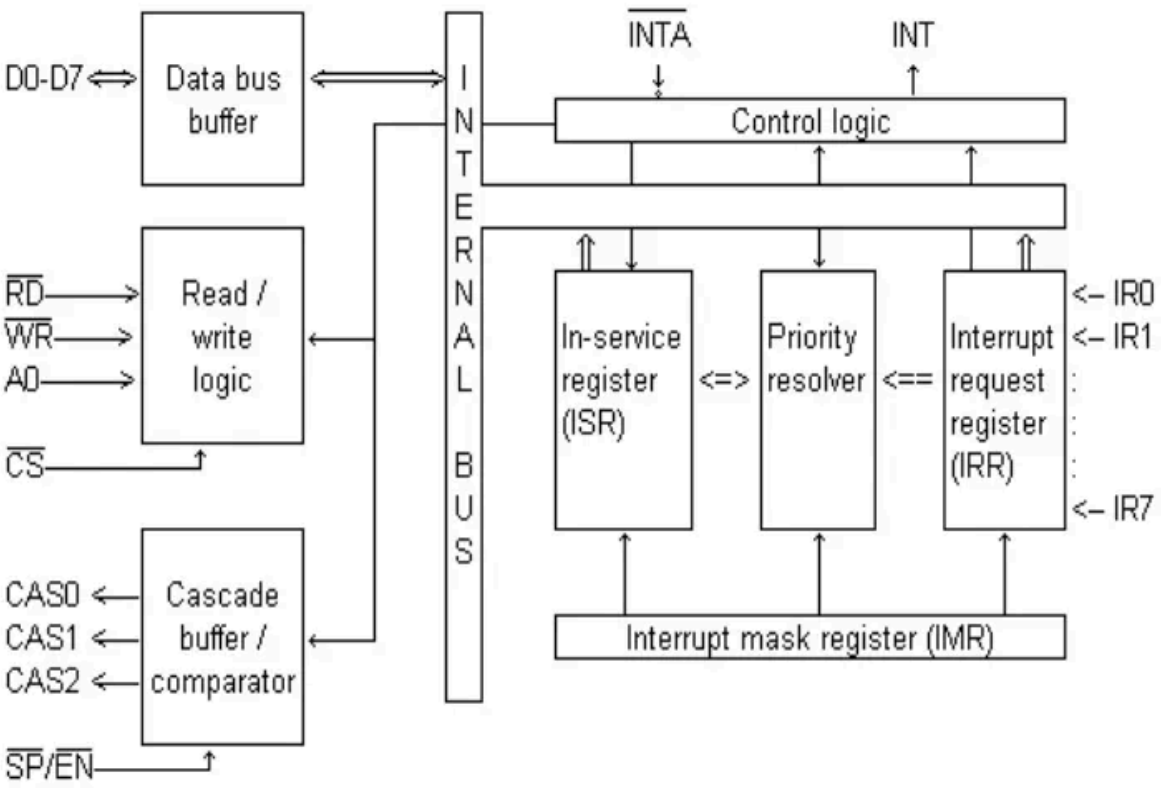
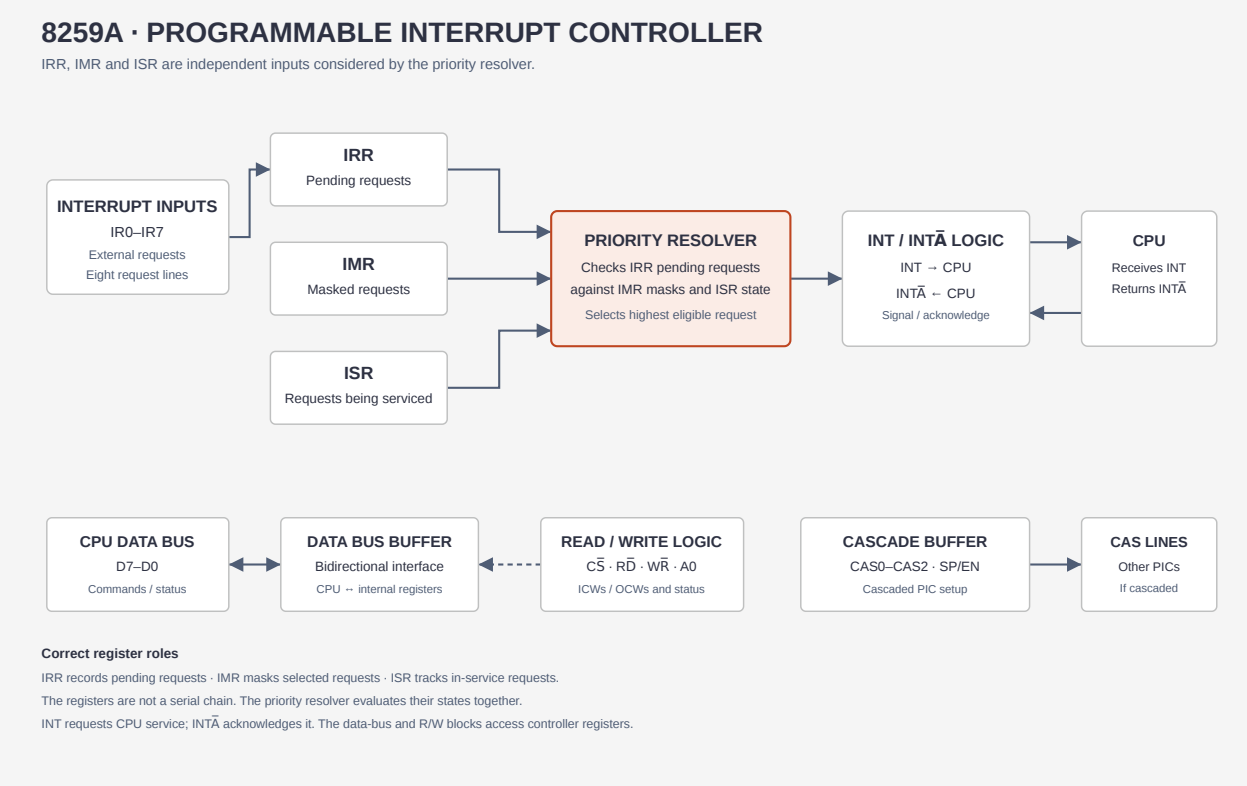


Fig 3.6 Block Diagram of 8259



Selected 8259A block diagram

Block	Function
IRR	Records incoming interrupt requests
IMR	Masks selected interrupt requests
ISR	Tracks requests currently being serviced
Priority Resolver	Selects highest-priority eligible request
Data Bus Buffer	Transfers command, status and data information
Read/Write Logic	Handles programming and read operations
Cascade Buffer/Comparator	Supports cascading multiple controllers

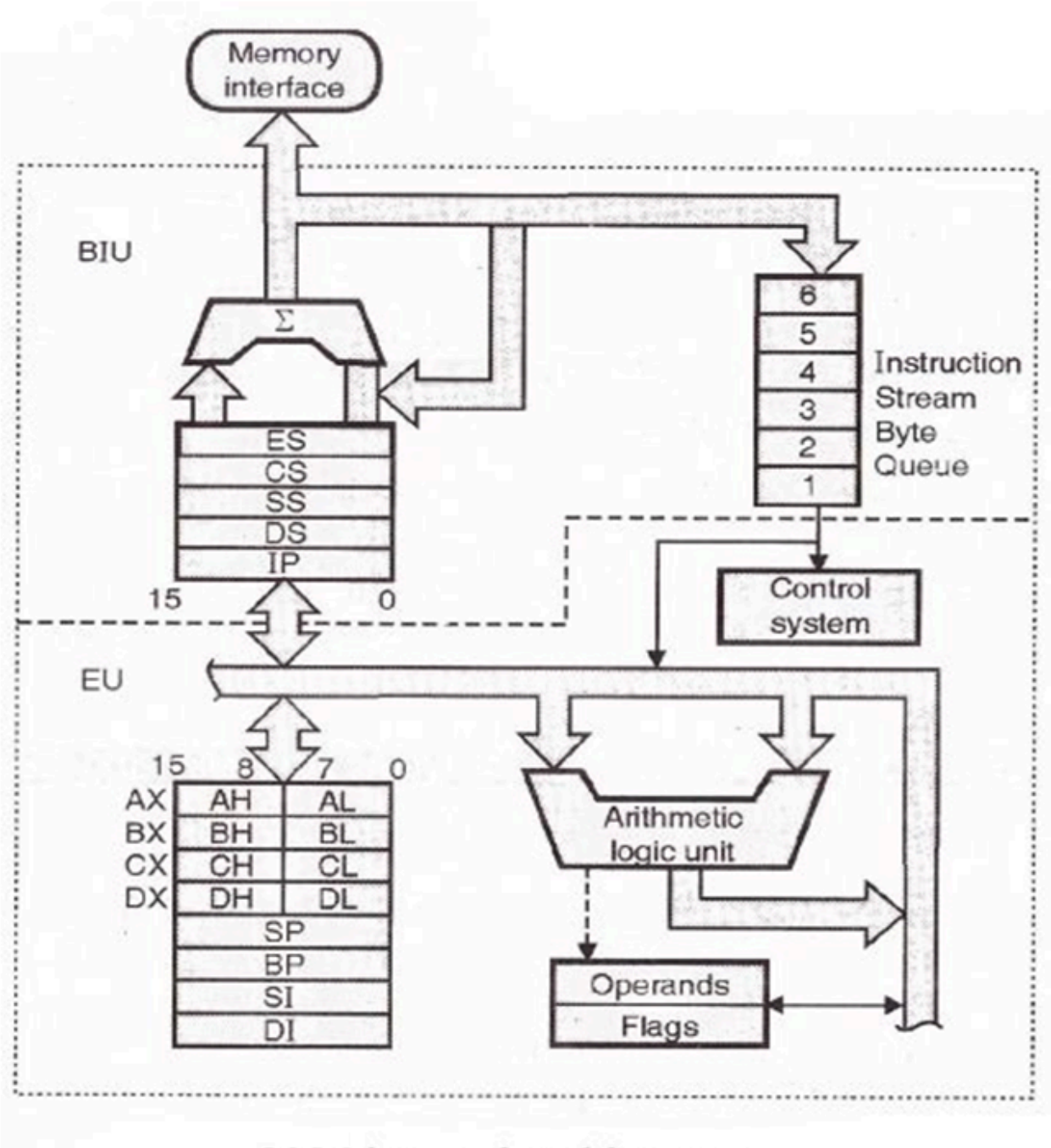
Ch. 8 - ADVANCED MICROPROCESSORS: 8086, 80286 AND 80386

1. Essential terminology

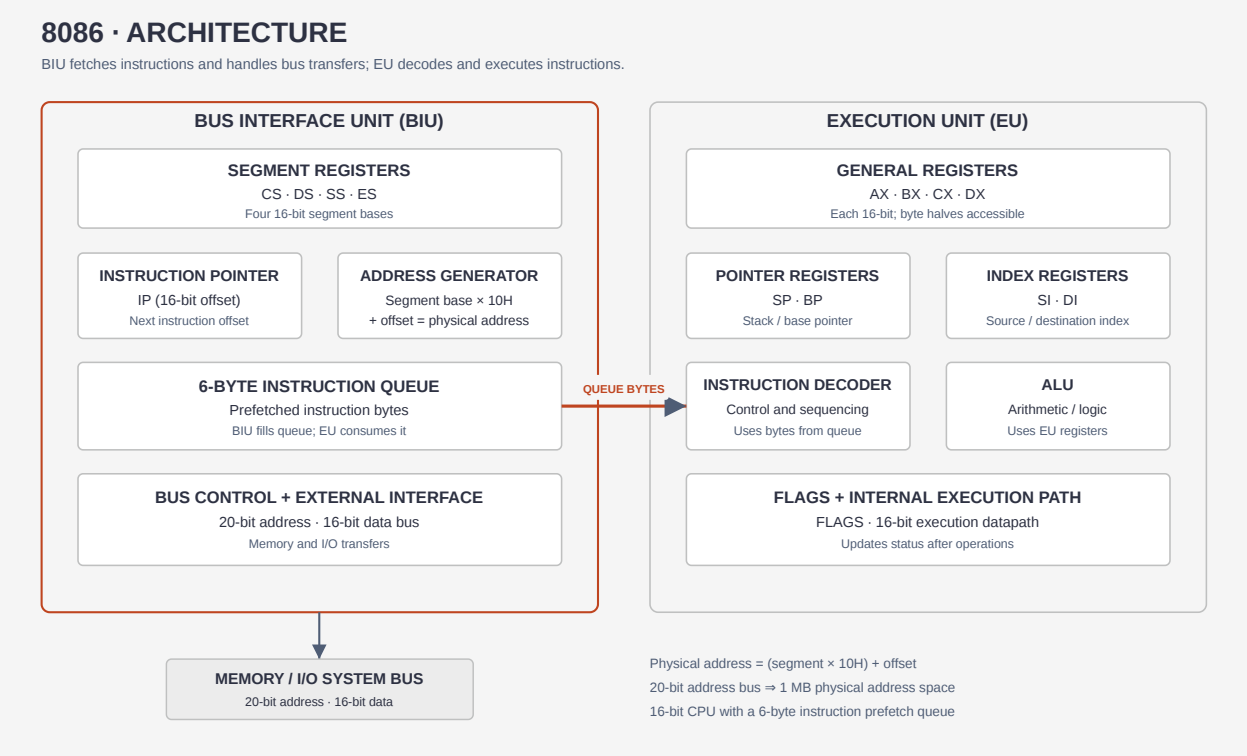
Term	Meaning
BIU	Bus Interface Unit; handles bus operations, instruction fetching and address generation in 8086
EU	Execution Unit; decodes and executes instructions
Segmentation	Dividing memory into logical segments
Physical address	Actual address used to access physical memory
Protected mode	Mode supporting memory protection and access controls
Descriptor	Data structure describing a segment or protected-mode object
Selector	Value identifying a descriptor
GDT / LDT	Global Descriptor Table / Local Descriptor Table
IDT	Interrupt Descriptor Table

2. M.IMP - 8086 architecture

- 16-bit processor.
- 20-bit physical address bus, allowing up to 1 MB physical address space.
- 16-bit data bus.
- 6-byte instruction prefetch queue.
- Two major units: BIU and EU.



8086 internal architecture



Selected 8086 BIU and EU architecture

- **BIU:** Fetches instructions, performs bus operations and generates addresses.
- **EU:** Executes instructions using ALU, registers and flags.
- Prefetching permits overlap between instruction fetching and execution when possible.

3. M.IMP - 8086 segmentation

Register	Purpose
CS	Code Segment
DS	Data Segment
SS	Stack Segment
ES	Extra Segment

Physical address formula

$$\text{Physical Address} = (\text{Segment} \times 10H) + \text{Offset}$$

Example: Segment = 2000H, Offset = 1234H.

$$2000H \times 10H = 20000H$$

$$20000H + 1234H = 21234H$$

The segment base is shifted left four bits before adding the offset.

4. M.IMP - Minimum mode vs maximum mode

Minimum mode	Maximum mode
MN/MX* is HIGH	MN/MX* is LOW
Intended for single-processor system	Supports multiprocessor configurations
8086 directly supplies main control signals	External bus controller such as 8288 generates bus commands
Simpler bus control	Additional bus-status and arbitration logic

5. 80286 vs 80386

Feature	80286	80386
Processor generation	16-bit	32-bit
Data size	16-bit	32-bit
Physical address width	24-bit	32-bit
Physical address-space capacity	16 MB	4 GB
Protected mode	Supported	Supported
Major distinction	Protected-mode memory management beyond 8086 model	Extended architecture to 32-bit registers and address handling

6. 80386 programming model and segments

The 80386 extends general-purpose registers to 32 bits: EAX, EBX, ECX, EDX, ESI, EDI, EBP and ESP, along with EIP and EFLAGS.

Its segment-register set includes CS, DS, SS, ES, FS and GS.

Segment descriptors and selectors

- A segment descriptor contains segment base, limit and access attributes.
- GDT contains global descriptors; LDT contains local descriptors.
- IDT contains interrupt and exception gate descriptors.
- A selector identifies a descriptor using its index, table indicator (TI) and requested privilege level (RPL).

For descriptor-table questions, identify the table, explain descriptor fields at a high level, and describe how the selector refers to an entry.

FINAL REVISION CHECKLIST

Priority	Topic
M.IMP	8085 internal architecture and block diagram
M.IMP	All five 8085 flags and set/reset conditions
M.IMP	Hardware/software interrupts, priority and vector addresses
M.IMP	All five addressing modes with examples
M.IMP	8255A block diagram, modes, control word and BSR
M.IMP	Memory-mapped vs I/O-mapped I/O
M.IMP	1-byte and 2-byte timing diagrams; instruction/machine cycles/T-states
M.IMP	8259A block diagram and functions
M.IMP	Ascending/descending sort and largest/smallest ALPs
M.IMP	8085 programming model, register pairs and RIM/SIM
M.IMP	EPROM/RAM interfacing, 74LS138 decoder and demultiplexing
M.IMP	8085 pin diagram and key pin functions
M.IMP	8086 architecture, BIU/EU, segmentation and physical-address calculation
M.IMP	80286/80386 architecture, programming model and protected-mode concepts
IMP	Arithmetic instructions, instruction formats, 16-bit operations and BCD conversion
IMP	Stack, CALL/RET, delay loops and block-transfer programming